

## Lab 4 - Building a Simple Processor Flow Control Instructions

In this lab series you will be designing a simple 8-bit single-cycle processor which includes an ALU, a register file and control logic, using Verilog HDL. Follow the guidelines given here to build your processor.

The microarchitecture of a processor is designed based on an Instruction Set. Your processor should implement the instructions `add`, `sub`, `and`, `or`, `mov`, `loadi`, `j` and `beq`. All instructions are of 32-bit fixed length, and should be encoded in the format shown below.

| OP-CODE<br>(bits 31-24) | RD / IMM<br>(bits 23-16) | RT<br>(bits 15-8) | RS / IMM<br>(bits 7-0) |
|-------------------------|--------------------------|-------------------|------------------------|
|-------------------------|--------------------------|-------------------|------------------------|

- Bits (31-24) : OP-CODE field identifies the instruction's operation. This should be used by the control logic to interpret the remaining fields and derive the control signals.
- Bits (23-16) : A register (RD) to be written to in the register file, or an immediate value (jump or branch target offset).
- Bits (15-8) : A register (RT) to be read from in the register file.
- Bits (7-0) : A register (RS) to be read from in the register file, or an immediate value.

Here are some examples about the usage and descriptions of these instructions:

```
add 4 1 2 (add value in register 2 to value in register 1, and place the result in register 4)
sub 4 1 2 (subtract value in register 2 from the value in register 1, and place the result in register 4)
and 4 1 2 (perform bit-wise AND on values in registers 1 and 2, and place the result in register 4)
or 4 1 2 (perform bit-wise OR on values in registers 1 and 2, and place the result in register 4)
j 0x02 (jump 2 instructions forward from the next instruction to be executed, by manipulating the
Program Counter. Ignore bits 15-0)
beq 0xFE 1 2 (if values in registers 1 and 2 are equal, branch 2 instructions backward by manipulating
the Program Counter)
mov 4 1 (copy the value in register 1 to register 4. Ignore bits 15-8)
loadi 4 0xFF (load the immediate value 0xFF to register 4. Ignore bits 15-8)
```

You will be building your processor in four steps:

- In Lab 2: part 1, you will build an 8-bit ALU which implements all the functional units required to support the instructions `add`, `sub`, `and`, `or`, `mov`, and `loadi`.
- In Lab 2: part 2, you will implement a simple 8x8 register file.
- In Lab3, you will implement the control logic and integrate all the components from parts 1 and 2 together to work as a complete processor.
- **In Lab 4, you will upgrade your processor to support `j` and `beq`.**

## Flow Control Instructions

Now that you have a working CPU which supports `add`, `sub`, `and`, `or`, `mov`, and `loadi` instructions, it's time to add microarchitectural support for the flow control instructions `j` and `beq`. For this, you will need to modify your top-level *cpu* and *alu* modules.

The *alu* needs an additional output port (`ZERO`) to indicate whether the result of a subtract operation is zero or not, in order to implement the `beq` instruction.

The *cpu* needs a new adder which can be used to compute the branch/jump target address based on the next PC value and the offset provided by the branch/jump instruction. We will assume that the new adder has a latency of two time units (`#2`), which will work in parallel to the ALU. The control functionality within the top-level *cpu* module needs to be modified to:

- manipulate the Program Counter using the immediate jump/branch target offsets provided in `j` and `beq` instructions (you will need to use adders and multiplexers);
- and generate the additional control signals required.

The timing diagrams for `j` and `beq` instructions are shown in Figure 5 below.

`j`:

|           |                         |  |                          |  |
|-----------|-------------------------|--|--------------------------|--|
|           |                         |  |                          |  |
| PC Update | Instruction Memory Read |  | Decode                   |  |
| #1        | #2                      |  | #1                       |  |
|           | PC+4 Adder              |  | Branch/Jump Target Adder |  |
|           | #1                      |  | #2                       |  |

`beq`:

|           |                         |  |                          |                 |
|-----------|-------------------------|--|--------------------------|-----------------|
|           |                         |  |                          |                 |
| PC Update | Instruction Memory Read |  | Register Read            | 2's Comp    ALU |
| #1        | #2                      |  | #2                       | #1    #2        |
|           | PC+4 Adder              |  | Branch/Jump Target Adder |                 |
|           | #1                      |  | #2                       |                 |
|           |                         |  | Decode                   |                 |
|           |                         |  | #1                       |                 |

**Figure 1: Timing Details for the Datapath**

Before you go and modify your code, first **draw a complete block diagram** of the datapath and control by extending lab 3 with the added components. Make sure to keep copies of your original files before modifying.

1. Modify the top-level *cpu* module and *alu* module to support the *j* and *beq* instructions. Include **a lot of comments**.
2. Write a testbench and thoroughly test your upgraded design. Hardcode your software program (instruction sequence) within the testbench file.
3. Submit a compressed file ***groupXX\_lab4.zip*** containing your Verilog files with the upgraded top-level *cpu module*, *alu* and *reg\_file* modules and any other Verilog files with any sub modules of your design, testbench, complete block diagram, and screenshots of timing diagrams clearly showing the synchronized operation of the datapath and control signals for the two new instructions.

Make sure you **add a lot of comments** when coding.

**Note that any form of plagiarism will result in zero marks for the entire lab.**